

PROJECT REVIEW – 1 REPORT

Course Title: Database Management Systems

Project Title: Social Media Management System

1. Problem Statement & Requirement Analysis

1.1 Problem Statement

Modern social networks handle vast amounts of user-generated data every second. Storing this data without a proper structure leads to slow queries, data inconsistency, and missing relationships.

The goal of this project is to design a structured relational database schema for a **Social Media Management System (SMMS)**. The system efficiently handles users, posts, comments, likes, follows, messages, notifications, and admin moderation while ensuring data integrity.

1.2 System Scope

The backend schema supports core social media functions:

- **User Accounts:** Registration, login, profile updates, and status tracking.
- **Content:** Text and media posts with timestamps.
- **Social Connections:** Follow/unfollow mechanics, post likes, and comments.
- **Messaging:** Private user-to-user direct messages.
- **Notifications:** Automated alerts for user actions (likes, comments, new followers).
- **Admin Control:** Account suspension, content removal, and action logs.

1.3 Requirement Analysis

Functional Requirements

1. **User Management:**
 - Register with a unique username and email.
 - Store encrypted passwords safely.
 - Update bio, location, and profile picture URL.
 - Search for other users or posts by keywords.
2. **Posts & Content:**
 - Create, edit, and delete text/media posts.
 - View a personal feed containing posts from followed users.
3. **Interactions:**
 - Follow or unfollow other users.
 - Like/unlike posts (maximum 1 like per user per post).
 - Add comments under posts.
4. **Messaging & Alerts:**

- Send direct messages with read/unread statuses.
- Receive real-time notifications for interactions.

5. Admin Features:

- View total users and active posts.
- Deactivate abusive user accounts and remove flagged posts.
- Log all admin actions for auditing.

Non-Functional Requirements

1. **Data Integrity:** Strict Primary Key (PK) and Foreign Key (FK) rules with cascade options.
2. **Normalization:** Adheres to Third Normal Form (3NF) to avoid redundancy.
3. **Performance:** Indexed columns on frequent lookups (username, created_at, post_id).
4. **Security:** Hashed passwords and restricted admin rights.

2. Entity-Relationship (ER) Diagram & Specification

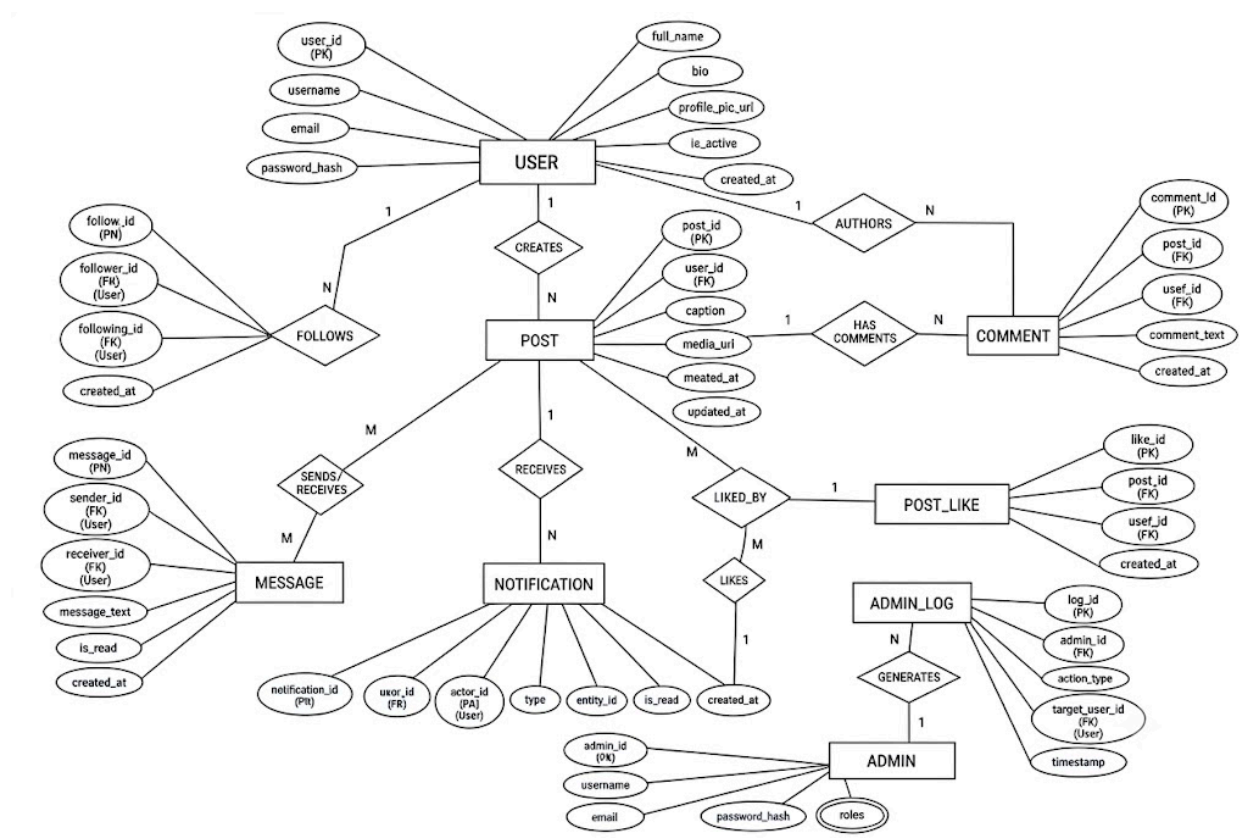
2.1 Entities & Attributes

1. **USER:** user_id (PK), username, email, password_hash, full_name, bio, profile_pic_url, is_active, created_at
2. **POST:** post_id (PK), user_id (FK), caption, media_url, created_at, updated_at
3. **COMMENT:** comment_id (PK), post_id (FK), user_id (FK), comment_text, created_at
4. **POST_LIKE:** like_id (PK), post_id (FK), user_id (FK), created_at
5. **FOLLOW:** follow_id (PK), follower_id (FK), following_id (FK), created_at
6. **MESSAGE:** message_id (PK), sender_id (FK), receiver_id (FK), message_text, is_read, created_at
7. **NOTIFICATION:** notification_id (PK), user_id (FK), actor_id (FK), type, entity_id, is_read, created_at
8. **ADMIN:** admin_id (PK), username, email, password_hash, role
9. **ADMIN_LOG:** log_id (PK), admin_id (FK), action_type, target_user_id (FK), timestamp

2.2 Cardinality Rules

1. **User to Post:** 1 to Many (One user can create multiple posts)
2. **Post to Comment:** 1 to Many (One post can have multiple comments)
3. **User to Comment:** 1 to Many (One user can make multiple comments)
4. **User to Post (Like):** Many to Many (Handled via POST_LIKE table)
5. **User to User (Follow):** Many to Many Recursive (Handled via FOLLOW table)
6. **User to User (Message):** Many to Many Direct (Handled via MESSAGE table)
7. **User to Notification:** 1 to Many (One user receives multiple notifications)
8. **Admin to Admin Log:** 1 to Many (One admin generates multiple log entries)

2.3 Structural ER Layout



3. Relational Schema & Normalization

3.1 Table Definitions

1. Users

- user_id: BIGINT, PK, Auto Increment
- username: VARCHAR(50), UNIQUE, NOT NULL
- email: VARCHAR(100), UNIQUE, NOT NULL
- password_hash: VARCHAR(255), NOT NULL
- full_name: VARCHAR(100), NOT NULL
- bio: TEXT, NULL
- profile_pic_url: VARCHAR(500), NULL
- is_active: BOOLEAN, DEFAULT TRUE
- created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

2. Posts

- post_id: BIGINT, PK, Auto Increment
- user_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
- caption: TEXT, NULL

- media_url: VARCHAR(500), NULL
 - created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
 - updated_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
3. **Comments**
- comment_id: BIGINT, PK, Auto Increment
 - post_id: BIGINT, FK -> Posts(post_id) ON DELETE CASCADE
 - user_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
 - comment_text: TEXT, NOT NULL
 - created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
4. **Likes**
- like_id: BIGINT, PK, Auto Increment
 - post_id: BIGINT, FK -> Posts(post_id) ON DELETE CASCADE
 - user_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
 - created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
 - Constraint: UNIQUE(post_id, user_id)
5. **Follows**
- follow_id: BIGINT, PK, Auto Increment
 - follower_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
 - following_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
 - created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
 - Constraints: UNIQUE(follower_id, following_id), CHECK (follower_id <> following_id)
6. **Messages**
- message_id: BIGINT, PK, Auto Increment
 - sender_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
 - receiver_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
 - message_text: TEXT, NOT NULL
 - is_read: BOOLEAN, DEFAULT FALSE
 - created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
7. **Notifications**
- notification_id: BIGINT, PK, Auto Increment
 - user_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
 - actor_id: BIGINT, FK -> Users(user_id) ON DELETE CASCADE
 - type: VARCHAR(20), NOT NULL
 - entity_id: BIGINT, NOT NULL
 - is_read: BOOLEAN, DEFAULT FALSE
 - created_at: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP
8. **Admins**
- admin_id: BIGINT, PK, Auto Increment
 - username: VARCHAR(50), UNIQUE, NOT NULL
 - email: VARCHAR(100), UNIQUE, NOT NULL
 - password_hash: VARCHAR(255), NOT NULL
 - role: VARCHAR(30), DEFAULT 'MODERATOR'
9. **Admin_Logs**

- log_id: BIGINT, PK, Auto Increment
- admin_id: BIGINT, FK -> Admins(admin_id)
- action_type: VARCHAR(100), NOT NULL
- target_user_id: BIGINT, FK -> Users(user_id), NULL
- timestamp: TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

3.2 Normalization Summary

- **First Normal Form (1NF):** All attributes contain single atomic values. Primary keys are defined for all tables.
- **Second Normal Form (2NF):** The schema is in 1NF. All non-key attributes fully depend on the primary key. Composite key issues are avoided by using surrogate IDs (like_id, follow_id) with unique constraints.
- **Third Normal Form (3NF):** The schema is in 2NF. No transitive dependencies exist (no attribute depends on another non-key attribute). For example, user details exist only in Users and are referenced elsewhere via user_id.4. Initial System Workflow

4.1 Onboarding & Feed Workflow

- **Registration:** User enters username, email, and password -> System validates uniqueness -> Hashed password stored in Users.
- **Login:** Credentials checked against Users.password_hash -> Session created.
- **Feed Display:** System fetches posts from Posts where user_id belongs to followed users, sorted by created_at DESC.

4.2 Content & Interaction Workflow

- **Creating a Post:** User submits text/media -> New row added to Posts.
- **Liking a Post:** User clicks like -> System inserts row into Likes -> Triggers a row in Notifications.
- **Commenting:** User types comment -> Row added to Comments -> Triggers notification to the post author.

4.3 Direct Messaging Workflow

- **Sending Message:** Sender posts message -> Inserted into Messages (is_read = FALSE).
- **Receiving Message:** Recipient opens chat -> System fetches unread messages and updates is_read = TRUE.

4.4 Admin Workflow

- **Moderation:** Admin logs in -> Deletes violating posts or sets Users.is_active = FALSE.
- **Cascading Cleanup:** Deleting a post automatically removes dependent Comments and Likes.
- **Logging:** Action saved in Admin_Logs with admin ID, action type, and timestamp.